Master Plan
Back-Analysis of the Models That Hurt the P&L

Stage 0 complete - Stages 1-3 planned, awaiting team agreement (Rule 1) - Commissioned by the human - 2026-10-21

Sources of truth: DATA_STORAGE.md (data census) - docs/postmortems/ (Stage 0 reports) - AGENTS.md Rule A6 (model governance) - JOURNAL.md Entries 9-11 (reasoning)

1. Mandate

When a model is disabled it must never influence a trade unless a human re-enables it, and every disable gets a post-mortem on the data that went in and why it failed. This plan is the systematic version of that rule, applied retroactively: a full back-analysis of every model that hurt the P&L, using the exact data each model consumed, producing per-model reports and tested fix proposals - with the GPU and the ATLAS agent leveraged where they genuinely help.

2. What Stage 0 found (the autopsy, done)

Nine per-model reports in docs/postmortems/, re-derivable from the append-only ledgers by running `python3 tools/pnl_autopsy.py`. The three findings that matter:

1. **Every sentinel-monitored model is underwater, and the guard is not guarding.** skim -19.96 bps/trip over 3,591 trips; scalp_fast -19.92 bps over 3,898; swing_turtle -$2,001 (-40% of $5,000 paper equity); multi_models -$13,353 (-267%); stats_models -$4,638 (-93%). Yet the sentinel still reports swing, multimodels and statsmodels all **live**, no reason recorded. Why it never parks them is itself a Stage-1 investigation. Per Rule A6, whether they stay live is the human's call.
2. **skim and scalp_fast are fee-machines, not alpha machines.** Mean gross +0.04 bps against 20 bps round-trip fees - 529x cost-to-capture - with a 2,023-loss streak on skim and time_cap exits dominating. Only trading less, holding longer, or a minimum-edge threshold at entry can rescue that.
3. **The fills engines mark their own homework.** grid/hft/movers exit on their own terms (profit_3s) and report ~100% win rates - scratch marks, flagged as upper bounds in every report. The trustworthy loss surface is the model ledgers.

Discipline note recorded in every report: when the whole book bleeds, every component bleeds - vote-attribution rankings are a search order for the Stage-1 counterfactuals, not verdicts.

3. The data estate (what the analysis runs on)

No database; everything is append-only JSONL (census in DATA_STORAGE.md):

- **scalp5/ledger/<model>.jsonl** - per-round-trip records: entry/exit, gross/net, fees, exit reason, and for the decision models the per-sub-model votes (multi_models, stats_models). This is the data that went in.
- **scalp5/state/fills.jsonl** - 12,503 engine fills with per-fill entry features (upper-bound marks).
- **scalp5/ledger/backend.jsonl** - 809,627 prediction records with full feature snapshots.
- **scalp5/state/sentinel.json** + archives - park/promote history with metrics.
- **backend/turtle_sessions.jsonl** - 4,310 Turtle session records (rejects included), now under the -data-dir anchoring; backend/ledger.jsonl and calibration.json currently empty (uncalibrated).

4. The program

Stage 0 - P&L autopsy (complete)

Per-model equity curves, drawdowns, expectancy, exit-reason and loss clustering, fee-vs-capture math, vote attribution, ranked hypotheses. Deliverables shipped: 9 reports + index + tool.

Stage 1 - Replay and counterfactuals (CPU; next)

Rebuild each losing trade's window from the ledgers; re-run exit logic under perturbed parameters; walk-forward splits. Every Stage-0 hypothesis names its perturbation: stop/target asymmetry sweeps; minimum-edge threshold at entry (expected move > k x fees); sub-model vetoes; session filters; symbol caps. **Deliverable:** backtest report with fixes ranked by expectancy delta, confidence interval and sample size; every result re-runnable from a committed command. **Also:** diagnose why the sentinel never parks models that have lost their equity.

Stage 2 - GPU-accelerated sweeps via ATLAS (conditional on a timing experiment)

Vectorize the Stage-1 replay in array semantics (JAX), then grid thousands of parameter combinations per model with walk-forward folds. Gated by the Rule-5 experiment: time one sweep window on device=cpu vs device=gpu; the ratio decides ATLAS's role (~3x stays local, ~30x ATLAS owns sweeps).

Stage 3 - Learned regime filters (only if the evidence demands it)

If Stage-1 counterfactuals show structure thresholds cannot express: a gradient-boosted model on CPU first (the dataset is small), a GPU neural net only as the last resort. The Go cmd/learner champion/challenger machinery and the empty calibration.json slot are the landing point - and under Rule A6 promotion is always a human act.

5. GPU & ATLAS - the architecture

The GPU is the fast path; the CPU is the audit path. Every GPU result must be re-runnable on CPU at reduced scale. ATLAS is a job runner behind a contract: the repo hands it a job (data arrays + parameter grid + exit-rule spec version + seed); it returns per-combo metrics plus provenance (git SHA, library versions, device). GPU outputs are proposals, never promotions (Rule A6).

The five rules, each with its exception named (Rule 5):

- **GPU fast / CPU audit.** *Exception:* floating-point non-associativity - equity accumulates in fp64 on both devices; agreement is within documented tolerance, not bit-equality.
- **Replay must reproduce the recorded ledger exactly before it sweeps anything.** *Exception:* entry-time indicators (e.g. RSI) replay from the recorded snapshot, not re-derived.
- **Branch-heavy exits (ratchets, trailing stops) break naive vectorization.** *Exception named up front:* masked segment logic or per-trade batching - built only if the timing experiment says the simple form is the bottleneck.
- **No result counts without provenance** (seed, spec version, device, library versions).
- **ATLAS relates to this repo like DeepSeek does:** files in, files out, channel for questions.

Why JAX and not CUDA-shaped code: identical code runs on CPU and GPU - development and the audit path use device=cpu, sweeps flip to device=gpu. Build the GPU agent collapses into write the array semantics, then flip a flag.

Honest sizing: the feature matrix (800k x ~20 float32 ~ 64 MB) is small; the GPU's value is throughput on sweeps and validation repeats, not capacity. If Stage 2's timing experiment disappoints, the plan stays

CPU-local and nothing else changes.

6. Governance

- **Rule A6:** a disabled model influences nothing; re-enabling is a human act, on the channel; no post-mortem, no disable; disables survive restarts and are audited.
- **Rule A1:** no feature is ever removed; fixes land alongside, behind flags, as proposals.
- **Lanes:** DeepSeek owns Go/model semantics and the math review; KIMI owns the harness, reports, and the repo; ATLAS owns GPU compute behind the contract; the human owns every promotion.
- **Rule 1:** Stages 1-3 build only after the team answers on the channel - silence is not agreement.

7. Acceptance criteria

stage	acceptance
1	Vectorized replay reproduces each model's recorded trades exactly (diffs explained); every proposed fix carries expectancy delta, CI, sample size, and a committed re-run command; sentinel non-parking diagnosed with evidence.
2	Timing experiment recorded (numpy vs JAX cpu vs gpu on one window); sweep results reproducible on CPU within tolerance; provenance attached to every row.
3	A learned model is justified only by Stage-1 evidence of non-linear structure; it enters as champion/challenger, never as an auto-promotion.

8. Immediate next actions

- **tools/export_npz.py** - parse the ledgers once into typed arrays (never re-parse JSONL).
- Vectorized NumPy replay of **skim** (simple exits, fee-dominated, huge n); acceptance = bit-match against its ledger.
- JAX port + the timing experiment (Rule 5) - decides ATLAS's role.
- Stage-1 counterfactual battery on the losing clusters, per the hypotheses in docs/postmortems/.
- **Human decision point:** the five bleeding models - stay live, park, or trial - each with its Stage-0 report in hand.

Everything in this plan re-derives from committed artifacts. If a number here and a ledger disagree, the ledger wins.